New Series

JavaScript: The New Parts

JavaScript programming language is the most popular implementation of the ECMAScript specification. Recently, version 6 was released. This new series explains all that you need to know about the new parts.

JS

ECMAScript 6

JavaScript has been around for more than two decades. The language has evolved over the years without breaking the Web, in spite of many browser wars. That the Web has not crashed is not a matter of luck, but through the coordinated efforts of the standards body named ECMA - European Computer Manufacturers Association — headquartered at Geneva.

The first version of JavaScript, then called Mocha, came out in March 1996 as part of Netscape Navigator 2.0. The creator, Brendan Eich, had created it in just 10 days! Now compare it with other programming languages like C and Java, which took four to five years. In 1997, Netscape handed over the language specifications for standardisation to ECMA. Though its commonly known as JavaScript, the official name accepted by the standards body is ECMAScript. Standardisation was essential for other browser vendors to also implement the language to keep the Web open. Companies like Microsoft, though, wanted a proprietary flavour of scripting in their browsers. The first version of the specification was released in June 1997. The third version, which was released in December 1999, was of great significance. The ambitious version 4 was abandoned and those features are later brought out as part of the current version 6 (Harmony).

Understanding the many names for the current version

It took six years for ECMAScript 2015 to become standard. Initially, this version was code named ES.next, also referred to as Harmony. Another short name, ES6 is also popular. But the current version is finally being called ECMAScript 2015. Why? There is an interesting reason. Considering the pace at which the Web is evolving, six years is an epoch by web timelines (the last version came out in 2009), to wait for a new version of the language that fuels the Web. The ES6 version did not seem to have a definitive timeline. With the intention of being more time-bound with releases, the final standard has been named ECMAScript 2015. Going forward, the year will be used to denote versions. The good news is that ECMAScript 2016 and ECMAScript 2017 are already in the pipeline. What about browser support? Can browser vendors keep pace with new versions? Yes, that is another very important aspect. All major browser vendors, including Microsoft, are ready to release features at that pace.

In the rest of this article, I will use JavaScript to denote ECMAScript. I will also use the short form ES6 to refer to ECMAScript 2015, for the sake of brevity. But bear in mind that JavaScript is language implementation and ECMAScript is language specification. Going through the specifications could be a daunting task. The specification is more user friendly for compiler developers than for casual JavaScript programmers. This article is to simplify and explain the new features.

The direction in which the language is evolving

Before we go into the nitty-gritty of what the new version of JavaScript has, let us understand what the major goals were, while rolling out the language.

- To make writing complex applications easier
- To make writing libraries easier
- The development of code generators like compilers, translators, transpilers and IDEs
- To always remain backward compatible and not break any existing Web pages or applications

New themes

The new version of JavaScript is a significant update in the last decade. There are a number of good features borrowed from different languages such as Python, C# and Java. It has dozens of major themes and hundreds of new features. For a comprehensive list, refer to the *Kangex* site in the *References* section.

Themes include fixing bad parts, introducing high-level programming language features, programmatic conveniences, optimisation for machine level languages, libraries for functional programming, syntax of object-oriented programming, allowing multiple libraries and frameworks to be used without conflict, etc.

The most criticised part of JavaScript is the scope of variables. There is only the function scope and global scope with the var keyword. Brendan Eich, the creator of JavaScript, attributes this to the short time given to him to create the first